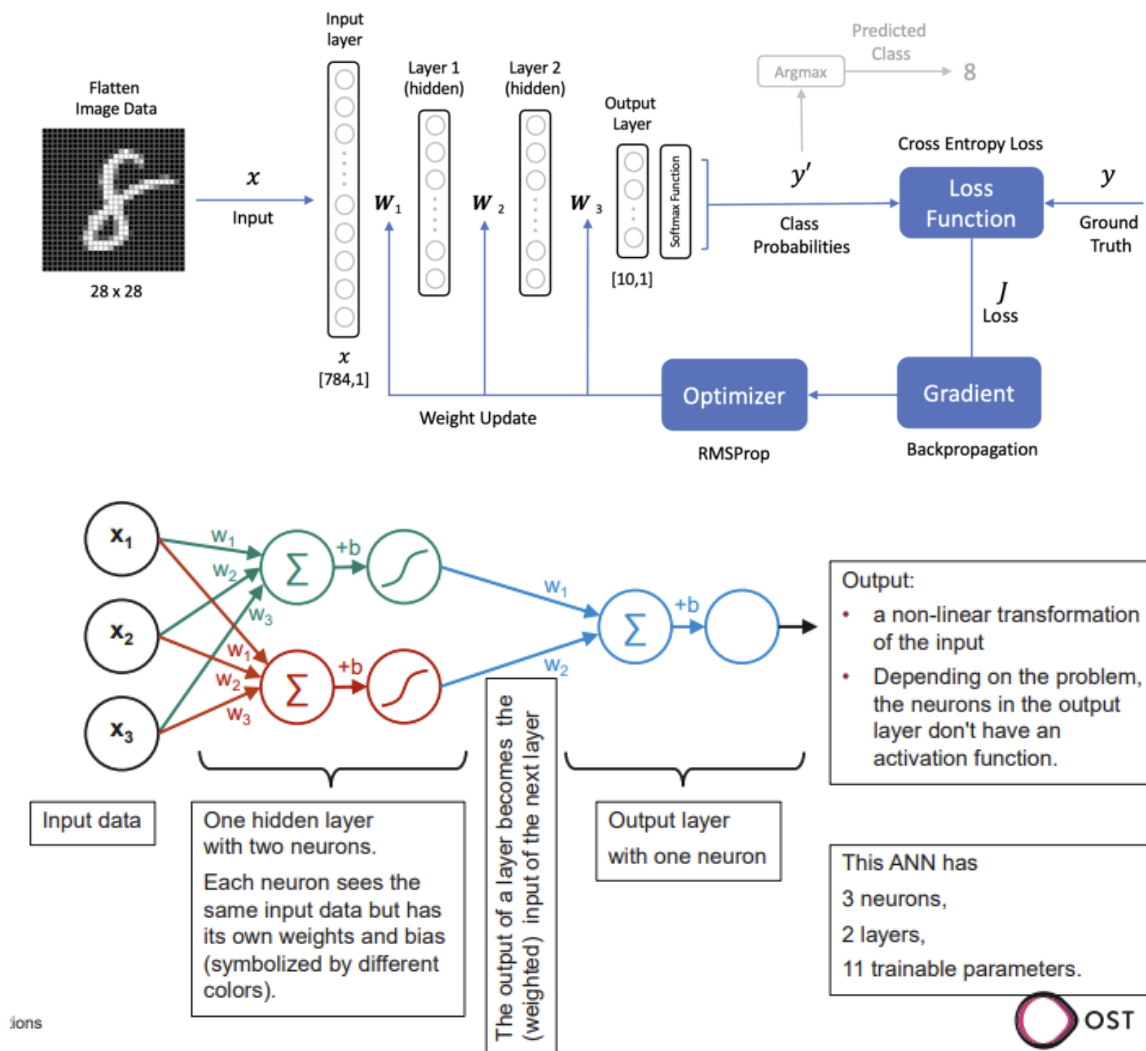




15. LÖSUNGEN PRÜFUNGEN

15.1. HS 2021

15.1.1. Aufgabe 1: Statistik und Zufallsvariablen (Bayes)

a) Wie gross ist die Wahrscheinlichkeit, dass eine zufällig ausgewählte Person einen Blutwert zwischen 51 bis 100 hat und gesund ist, also $\Pr(X=3, Y=1)$? **0.1**

c) Wie gross ist die Wahrscheinlichkeit, dass eine zufällig ausgewählte Person gesund ist? **0.6**

d) Wie gross ist die Wahrscheinlichkeit, dass eine zufällig ausgewählte Person einen Blutwert zwischen 21 und 100 hat? **0.7**

e) Von einer Person weiss der Arzt mit Sicherheit, dass sie gesund ist. Berechnen Sie die bedingte Wahrscheinlichkeit, dass der Blutwert zwischen 51 und 100 liegt.

- Verwenden Sie Werte aus der Tabelle und die Definition $\Pr(X,Y) = \Pr(X|Y) * \Pr(Y)$.
- Notieren Sie die Formel mit den Zahlenwerten, nicht nur das Ergebnis.
- Lassen Sie ungerade Zahlen als gekürzten Bruch stehen.

$$\Pr(X=3 | Y=1) = \Pr(X=3, Y=1) / \Pr(Y=1) = 0.1 / 0.6$$

f) Berechnen Sie die bedingte Wahrscheinlichkeit, dass eine Person gesund ist, wenn der Blutwert zwischen 51 und 100 liegt.

- Verwenden Sie Bayes Rule und das Ergebnis aus der obigen Aufgabe.
- Notieren Sie die Formel mit den Zahlenwerten, nicht nur das Ergebnis.

$$\Pr(Y=1 | X=3) = \Pr(X=3, Y=1) / \Pr(X=3) = 0.1 / 0.4$$

15.1.2. Aufgabe 2: SGD und Regularisierung

Sie wollen das folgende lineare Modell auf Daten fitten: Modell: $\hat{y}_i = ax_i + b$ Das Modell wird mit SGD optimiert. Die Loss-Funktion setzt sich zusammen aus dem Mean Squared Error (MSE, mit Vorfaktor 1/2) plus einem L-1 Regularisierungsterm (aka Lasso-Regression) mit dem Regularisierungsparameter λ .

a) Notieren Sie hier die vollständige Loss-Funktion (MSE und Regularisierung)

		Blutwert 0-20	Blutwert 21-50	Blutwert 51-100	
		X=1	X=2	X=3	
gesund	Y=1	0.3	0.2	0.1	0.6
krank	Y=2	0	0.1	0.3	0.4
		0.3	0.3	0.4	1

i. Schreiben Sie die Loss-Funktion als Summe für N Datenpunkte.

ii. Schreiben Sie die Formel so aus, dass klar ersichtlich ist, wie die Daten x_i , y_i sowie die Parameter a und b in die Berechnung einfließen.

$$\frac{1}{2N} \sum_{i=1}^N (y^i - (ax^i + b))^2 + \lambda (|a| + |b|)$$

b) Zum Zeitpunkt t ist der Parametervektor $[a_t = +2, b_t = -3]$. Es wird ein Optimierungsschritt mit dem Datenpunkt $(x_i = 1, y_i = 4)$ ausgeführt. Der Regularisierungsparameter hat den Wert $\lambda = 0.1$. Berechnen Sie den Loss. Das heisst, werten die obige Formel mit den gegebenen Zahlen aus. Notieren Sie nicht nur das Endergebnis, sondern die ganze Berechnung.

$$1/2 \cdot (4 - (2 \cdot 1 - 3))^2 + 0.1 (2 + 3) = 13$$

c) Angenommen wir verwenden das obige Beispiel, aber mit $\lambda = 10^9$ (sehr gross im Vergleich zu den anderen Werten, der genaue Wert spielt keine Rolle), begründen Sie, ob die folgenden Aussagen zutreffen oder nicht. (Es ist keine Berechnung nötig um die Frage zu beantworten):

i. Das Optimum liegt praktisch bei $(a_{opt} \approx 0, b_{opt} \approx 0)$.

RICHTIG: Bei sehr grossem λ dominiert der Regularisierungsterm $\lambda(|a| + |b|)$. Das Minimum der Loss-Funktion liegt daher nahe bei $a = 0$ und $b = 0$, unabhängig vom MSE-Term.

ii. Die Parameter a und b gehen in einem einzigen Optimierungsschritt praktisch gegen 0, also $(a_{t+1} \approx 0, b_{t+1} \approx 0)$.

FALSCH: SGD macht schrittweise Updates abhängig von der Lernrate. Auch bei sehr grossem λ werden a und b nicht in einem einzigen Schritt exakt 0, sondern nähern sich über mehrere Schritte an.

15.1.3. Aufgabe 3: Lineare Regression und Cross-Validation

Gegeben sind folgende drei Datenpunkte, auf welche eine Regressionsgerade $\hat{y}_i = ax_i + b$, angepasst wird.

a) Skizzieren Sie die optimale Regressionsgerade direkt in die obige Grafik. (keine Berechnung notwendig, nur graphisch).

b) Ergänzen Sie die Residuen, beschriften Sie sie mit r_1, r_2, r_3 .

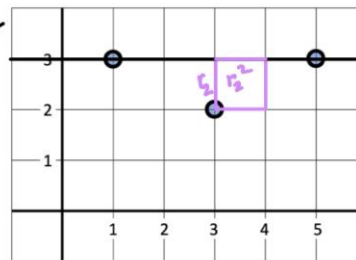
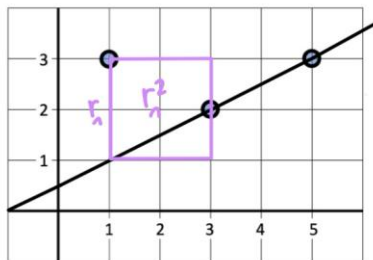
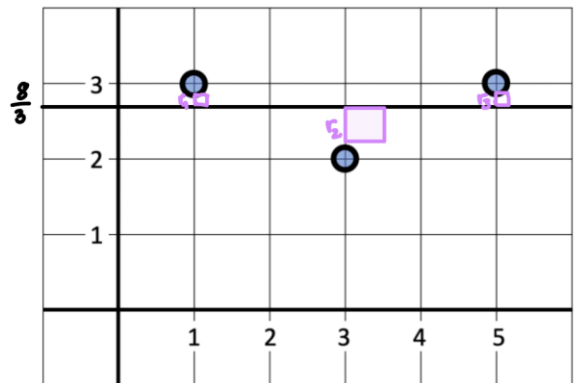
c) Direkt in der Skizze, visualisieren Sie die drei Fehlerquadrate r_i^2 . Führen Sie nun (graphisch) eine 3-fold Cross-Validation durch.

Verwenden für jeden der drei Splits je einen der drei vordruckten Graphen. In jedem Graphen:

d) Skizzieren Sie die Regressionsgerade durch die Trainings-Daten

e) Skizzieren Sie auf den Test-Daten r_i und r_i^2

f) Aus den drei Graphen, schätzen Sie den mittleren Test-Error der durch die 3-fold Cross-Validation ermittelt wird.



15.1.4. Aufgabe 3: Model evaluation and selection

Wir haben 2 Klassifikatoren, um E-Mails in "Spam" oder "Sicher" (safe) zu trennen. Der Klassifikator gibt 1 (positive) aus, wenn die E-Mail als Spam klassifiziert wird.

a) Berechnen Sie Precision and Recall für Klassifikator #1

$$TP = 3$$

$$FP = 3$$

$$\text{recall} = TP / (TP + FN) = 3 / 3 = 1$$

$$\text{precision} = TP / (TP + FP) = 3 / (3 + 3) = 3 / 6 = 1 / 2$$

b) Berechnen Sie Precision and Recall für Klassifikator #2.

$$TP = 2$$

$$FP = 1$$

$$TN = 3$$

$$\text{recall} = TP / (TP + FN) = 2 / 2 = 1$$

$$\text{precision} = TP / (TP + FP) = 2 / (2 + 1) = 2 / 3$$

c) Ihr Spamfilter löscht automatisch die als Spam klassifizierten E-Mails. Welchen Klassifikator würden Sie für Ihre E-Mail-Klassifizierung bevorzugen? Begründen Sie Ihre Wahl. **Klassifikator 2, weil bei Klassifikator 1 potenziell wichtige Mails gelöscht werden könnten.**

Mail ID	Email Contents	Klasse/Label	Klassifikator#1	Klassifikator#2
1	Hurry Sale Tomorrow	Spam (=1)	1 TP	1 TP
2	Rain tomorrow	Spam	1 TP	1 TP
3	Sale price tomorrow	Spam	1 TP	0 FP
4	tomorrow workshop rain	Safe (=0)	1 FP	0 TN
5	tomorrow cake	Safe	1 FP	0 TN
6	Party beer	Safe	1 FP	0 TN

15.1.5. Aufgabe 4: K-NN

Wir haben den folgenden Datensatz. Jeder Datenpunkt hat zwei Features, x_1 und x_2 , sowie eine Klassenbezeichnung (labels) Spam (+) und sicher (-).

x_1	x_2	spam/safe	Platz für Berechnung Datenpunkt (-0.5, -0.5)	Platz für Berechnung Datenpunkt (1.0, 1.0)
-1	0	+	$0.5 + 0.5 = 1$	$2 + 1 = 3$
0	-1	-	$0.5 + 0.5 = 1$	$1 + 2 = 3$
1	0	-	$1.5 + 0.5 = 2$	$0 + 1 = 1$
0	1	+	$0.5 + 1.5 = 2$	$1 + 0 = 1$
1	2	+	$1.5 + 2.5 = 4$	$0 + 1 = 1$
1	3	+	$1.5 + 3.5 = 5$	$0 + 2 = 2$

a) Datenpunkt: $x_1 = -0.5$, $x_2 = -0.5$, $k = 2$, Distanz = Manhattan → kann beides sein (Spam/Safe)

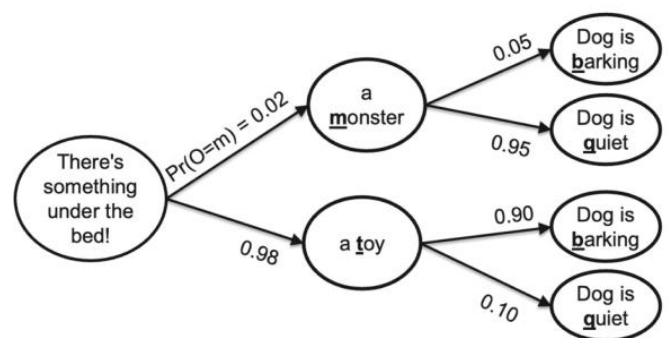
b) Datenpunkt: $x_1 = 1$, $x_2 = 1$, $k = 3$, Distanz = Manhattan → + (spam)

15.2. HS 2023

15.2.1. Aufgabe 1: Bayes Rules

a) You come home late and it's suspiciously quiet. Use Bayes' rule to calculate the probability that there is a monster under the bed given the dog is quiet. (6 Pts)

- First write the formula without inserting values, then calculate.
- Use correct notation (same pattern as used above to annotate the first arrow)
- Use the random variable O for Object. O takes the values m or t .
- Use the random variable D for Dog. D takes the values b or q .



$$P(O = m \mid b = q) = (P(b = q \mid O = m) \cdot P(O = m)) / P(D = q) = (0.95 \cdot 0.02) / (0.98 \cdot 0.1 + 0.02 \cdot 0.95) = 0.16$$

15.2.2. Aufgabe 2: Model Fitting

Scenario: Consider a dataset with $N = 5000$ samples, each sample with $m = 100$ features. A polynomial regression model of order $p = 200$ is trained on this dataset. We observe that the training error is low. The model performs poorly during validation.

a) Model fit: Overfit

b) Bias-Variance Trade-off:

The model is probably too complex, meaning it has low bias and high variance. The polynomial order $p = 200$ fits the training set very well, but small changes in the data lead to large changes in the fitted model.

→ Poor generalization on validation data.

c) Regularization choice:

L1 regularization (Lasso) is optimal because it drives some weights exactly to 0. This removes irrelevant or redundant features and reduces model complexity.

d) Cost function with regularization term:

$$\text{Loss} = \frac{1}{2N} \sum_{j=1}^N (y_j - \hat{y}_j)^2 + \lambda \cdot \sum_{j=1}^p |w_j|$$

15.2.3. Aufgabe 3: Working with Data in Python

In the class exercises, we used a powerful library to read, analyze and manipulate data. Look at the following line of Python code. It reads a csv file. `df = pd.read_csv('myfile.csv')`

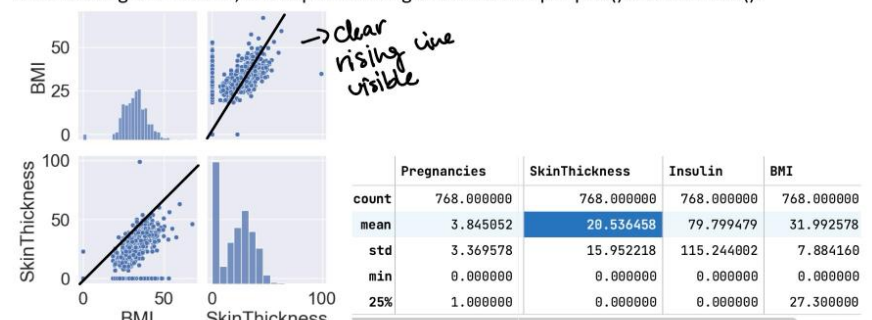
a) Which Python package is used here to read the csv file? **Pandas**

b) What is the datatype of the object returned by `read_csv`? **Pandas DataFrame**

c) A data analyst says: "BMI and skin thickness are correlated". Explain the statement and annotate the figure to visualize what the data analyst is saying. No calculation needed. This means that if the BMI increases, the skin thickness also increases.

The cloud of points forms a clear upward-sloping pattern.

After loading the dataset, it is explored using the functions `pairplot()` and `describe()`:



d) The `describe()` function calculates a mean "SkinThickness" of ~20.53. The data analyst tells you: "There is something wrong with the dataset. The mean "SkinThickness" should be around 30". Explain why the data analyst is right. According to the table, around 25% of the values are 0, which indicates that we might have many missing values that were encoded as 0. These zeros pull the mean down, which explains why the average SkinThickness is around 20 instead of the expected value around 30.

15.2.4. Aufgabe 4: Word Representations and Cosine Similarity

Write the formula for the Cosine-Similarity for the simplified case where all vectors are normalized (length of each vector = 1). $\cos(\Theta) = x \cdot y$. What does the symbol Θ in the above formula refer to? The angle between the two vectors x and y . It measures how similar the directions of the two vectors are.

In terms of cosine similarity, is `word_3` more similar to `word_1` or to `word_2`? Do a calculation to justify your answer. The closer to 1 the more similar they are. Dot product of 1 & 3: $0.284 \cdot 0.047 + 0.946 \cdot 0.977 + 0.158 \cdot -0.209 = 0.904$

15.2.5. Aufgabe 5: Adaboost

The above figure shows the decision boundary (line $x = 2.0$) after the first iteration of AdaBoost. We address the classifier after the 1st iteration as weak learner 1 (WL1). The WL1 predicts points to the left of the boundary as positive and those to the right as negative. Answer the following questions based on the above information.

a) Draw the confusion matrix for WL1. Calculate accuracy and precision for WL1.

$$\text{accuracy} = (\text{TP} + \text{TN}) / \text{total} = (10 + 5) / 23$$

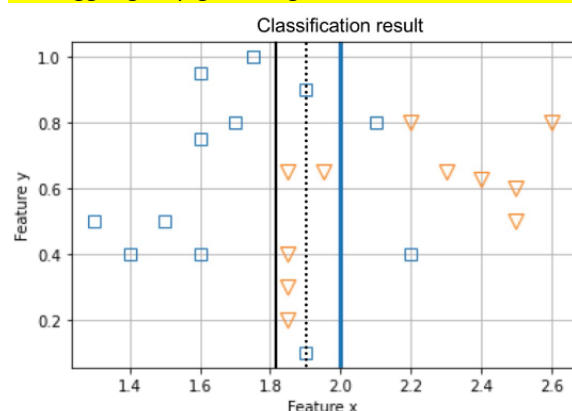
$$\text{precision} = \text{TP} / (\text{TP} + \text{FP}) = 10 / (10 + 5) = 10/15$$

	$\wedge^+$	$\wedge^-$
+	10	2
-	5	6

b) In the next iteration, Adaboost assigns new instance weights. In the graph below, highlight the points with higher instance weights using circles.

c) Draw a solid line that roughly shows the decision boundary for WL2 (weak learner after 2nd iteration). State your reasoning.

The bigger group gets weighted more and thus it shifts to the left



f) Adaboost is terminated after completing the 2nd iteration. An ensemble is created. Roughly draw a dashed line in the above plot as the ensemble's decision boundary based on the boundaries of WL1 and WL2. State your reasoning. The final ensemble is a decision between the two previous points

15.3. FS 2024

15.3.1. Aufgabe 1: Bayes

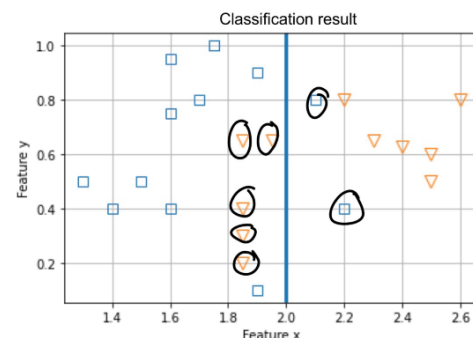
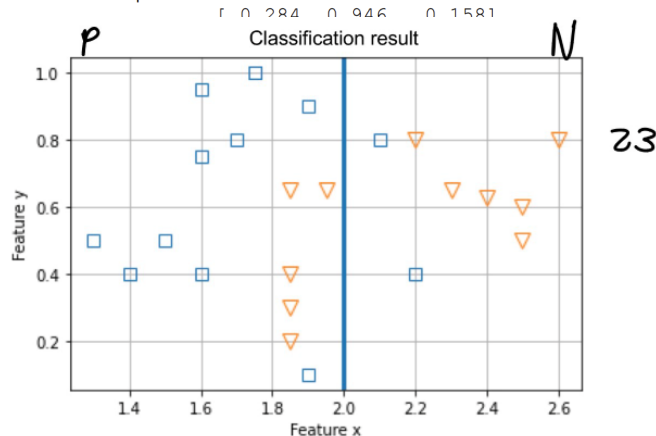
A bot performs some task on a computer. An Intrusion Detection Systems (IDS) is running to detect malware by analyzing internet traffic. Malware generates a different outbound traffic pattern than the bot. The following table shows the joint probabilities. Assume that either the bot or the malware is active, not both at the same time.

a) Calculate all marginal probabilities. Write the result directly into the table above.

Now consider the following Python script and its output:

```
1 import numpy as np
2
3 word_1 = np.array([ 0.9, 3, 0.5])
4 word_1 = word_1 / np.linalg.norm(word_1)
5 word_1 = np.round(word_1,3)
6 print(word_1)
7
8 word_2 = np.array([-2, 3.3, -0.5])
9 word_2 = np.round(word_2 / np.linalg.norm(word_2), 3)
10 print(word_2)
11
12 word_3 = np.array([0.2, 4.2, -0.9])
13 word_3 = np.round(word_3 / np.linalg.norm(word_3), 3)
14 print(word_3)
15
16 dot_product12 = np.dot(word_1, word_2)
17 print("dot_product word_1, word_2 = {}".format(dot_product12))
18 dot_product23 = np.dot(word_2, word_3)
19 print("dot_product word_2, word_3 = {}".format(dot_product23))
```

Output:



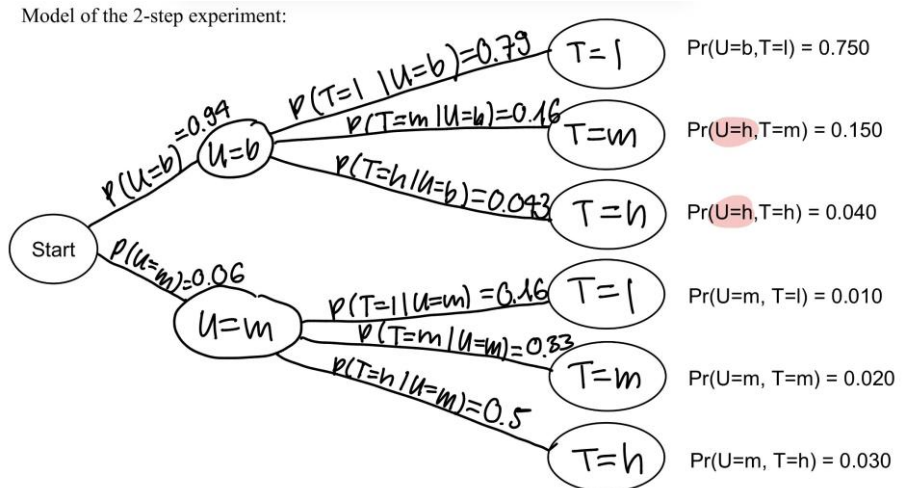
Pr(U,T)		outbound traffic (T)			
		T=low	T=medium	T=high	
user (U)	U=bot	0.750	0.150	0.040	0.94
	U=malware	0.010	0.020	0.030	0.06
		0.76	0.17	0.07	1

b) A colleague makes the following statement: "High traffic is a signature of the bot, not of the malware because the table shows 0.030 vs. 0.040.". Is he right or wrong? Explain! **Wrong: The table only shows joint probabilities, but the question asks for a conditional probability. The conditional probability also considers that bots are more frequent than malware.**

c) Model the outbound traffic as a two-step experiment. Complete the graph on the next page with the missing nodes and arrows. d) Inside each node write the event it represents. For example "T=h". e) Along each arrow, write an expression of the form $\Pr(\dots)=\dots$

f) Use Bayes' Rule to answer the question: "The Intrusion Detection Systems (IDS) observes high traffic. What is the probability, that a malware is active?"

$$\Pr(U=m \mid T=h) = \Pr(U=m, T=h) / \Pr(T=h) = 0.03 / 0.07$$



15.3.2. Aufgabe 2: Regularization

You want to predict a target variable y for some input feature x . The underlying unknown input function (generated from a polynomial of order 5) is partially shown by the gray line. Some of the observed noisy data points are shown in blue. The target prediction of the model is written as $\hat{y}$. You now train a polynomial regression model of order M for prediction using the following code snippet.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, shuffle=True)
poly = PolynomialFeatures(M, include_bias = False)
poly.fit_transform(X_train)
linear_regression = LinearRegression(fit_intercept = True)
linear_regression.fit(X_train, Y_train)
```

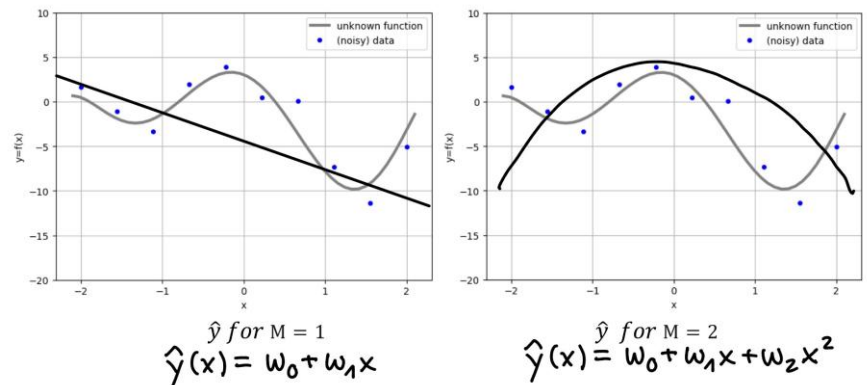
→ creates features $x, x^2, x^3, \dots$
 adds w_0
 apply linreg must be linear, that's why we use weights

a) Write the regression model formula corresponding to the above code. $\hat{y} = w_0 + \sum_{i=1}^M w_i x^i$

b) Write the error/loss function that your model tries to minimize. $E = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$

c) Sketch plausible plots for $\hat{y}$ for $M=1$ and $\hat{y}$ for $M=2$.

d) When asked for the best value of M , you choose the value of M to be 50. You argue that the training error and test error reduce monotonically as we increase M . Discuss. If M is large, we get an overfitting which means that we get a high variance and thus the test error increases. The training error reduces, and we get a near perfect fit. Low training error != good model.



e) You now choose a simple model with $M=1$ and a more complex model with $M=20$. You run the model training pipeline 5 times. This results in 5 trained models, each for $M=1$ and $M=20$. Let us call these models as $m_1, m_1', \dots$ for $M=1$ and $m_{20}, m_{20}', \dots$ for $M=20$. Hint: Notice that `train_test_split` shuffles the dataset and returns a different split each time – you train on a different split each time.

iii.) Explain the effect of model complexity M on bias and variance using your plots. Increasing model complexity reduces bias but increases variance. For $M=1$ the model underfits with high bias and low variance. For $M=20$ the model fits the data well but shows high variance. Averaging predictions reduces variance but does not reduce bias.

15.3.3. Aufgabe 3: Gradient Descent Optimization

We optimize a two-parameter model using Gradient Descent. The model and the dataset are not relevant. The MSE is used. The graph shows the "Loss Landscape", with black gridlines and contours overlayed for readability. Example Vectors are added to provide the scale. Solve the question using the graph; no calculations required.

a) Estimate the best possible parameters. Try to be as accurate as ± 0.5 units for the params, and ± 10 units for the Loss: $p1_{best} = 5$ $p2_{best} = -4.5$ $Loss_{best} = 10$

b) Graphically perform 2 update steps: 1. The optimizer is initialized with $[p1 = -2, p2 = -6]$, mark this with a cross and write "A" next to it. 2. Perform a Gradient Descent step in the given error landscape. Assume that the learning rate is such that the step-size is 2cm. Draw an arrow indicating this update step. 3. Perform a second update step. Assume the step-size is 1cm.

c) The optimizer is now reinitialized with $[p1 = +6, p2 = -6]$. Graphically perform one update step 1. Mark the start location with a cross and write "B" next to it. 2. Perform one Gradient Descent step. Assume the step-size is 4cm.

3. How does the Loss change? Explain the problem and how it could be solved. The learning rate (step size) is too large. It causes overshooting. Fixes: Reduce learning rate

15.3.4. Aufgabe 4: Neural Networks

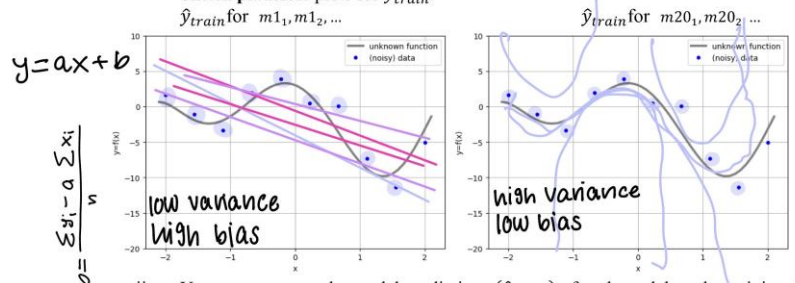
You are working with a neural network that classifies images of animals into one of four classes: cat, dog, bird, or fish. You are tasked with designing a simple feedforward neural network to classify grayscale images of size 4×4 pixels into these four classes.

1. Specify and draw your network showing: 2. Now you want to add an activation function to the output neurons of your network. Which activation function will you use? Explain why it is a suitable choice for this task. Softmax, because this is a 4-class classification problem and the outputs should form a probability distribution.

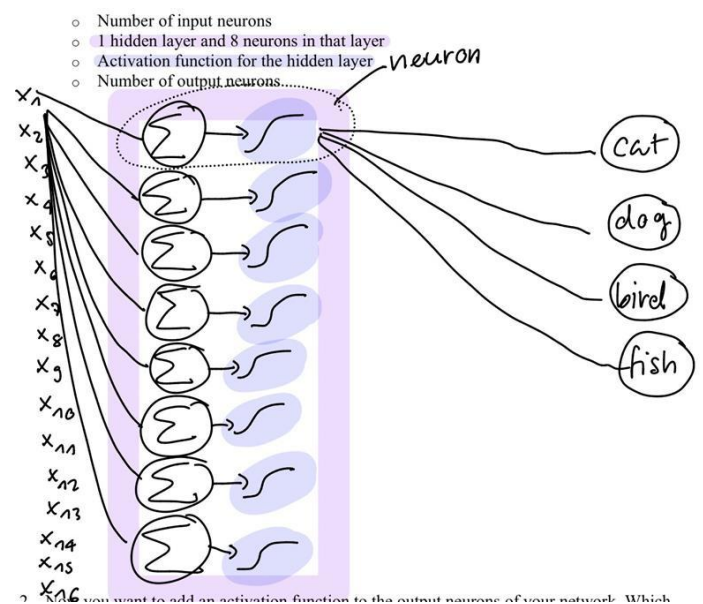
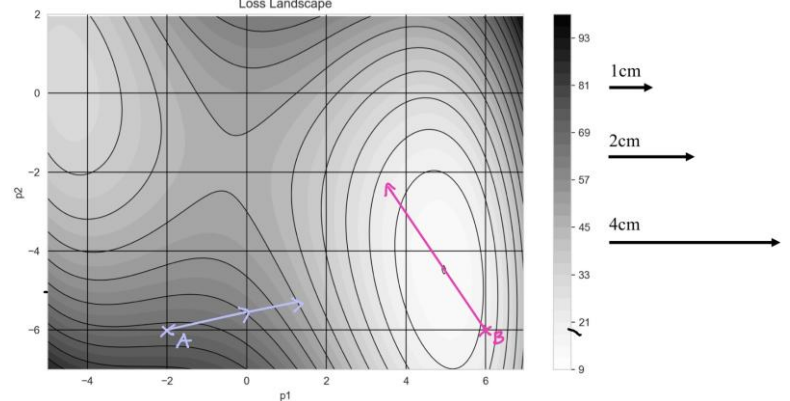
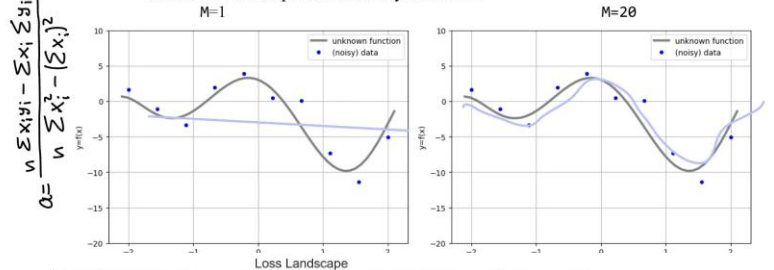
3. Calculate the total number of trainable parameters in your network, considering weights and biases. Show all the steps of your calculation. 1. Hidden layer: $8 \cdot (16+1)$, Output: $4 \cdot (8+1) \rightarrow$ total = 172 $\rightarrow$ formula: sum of all hidden layers + output (neurons $\cdot$ (weights + 1 bias))

4. Can adding polynomial features to logistic regression make it as expressive as a neural network with a single hidden layer? Justify your answer. Points will be awarded only if a justification is provided. Adding polynomial features increases the expressiveness of logistic regression and allows non-linear decision boundaries. However, it is still limited to polynomial functions with a fixed feature expansion. A neural network with one hidden

i. You now sketch the model predictions $\hat{y}_{train}$ of each model on the training data. Sketch plausible plots for $\hat{y}_{train}$.



ii. You now average the model predictions ($\hat{y}_{train}$) of each model on the training data. Sketch how these predictions may look like.



layer is more flexible and, according to the universal approximation theorem, can approximate arbitrary functions. Therefore, polynomial logistic regression is not as expressive as a neural network.

15.4. THEORY QUESTION 2021 PART 1

15.4.1. PMF, Gleichverteilte Zufallsvariable (uniform probability mass function)

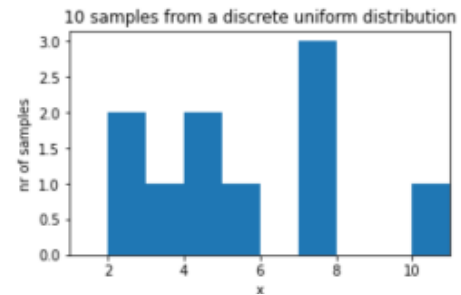
Angenommen Sie verwenden eine Uniform-Distribution um 10 ganzzahlige Zufallszahlen zwischen 1 und 10 zu erzeugen

- Skizzieren Sie ein qualitatives, plausibles Histogramm mit 10 Bins.
- Beschriften Sie den Graphen

-Ein Histogramm in welchem jeder Wert von 1 bis 10 genau 1 mal vorkommt wäre falsch (da extrem unwahrscheinlich).

-korrekte Beschriftungen. Titel, x-label, y-label. Die Position der x-ticks kann variieren, Pyplot macht sie am linken Rand des Balkens. Y-ticks wären besser Ganzzahlig

array([5, 7, 10, 4, 4, 2, 3, 7, 7, 2])

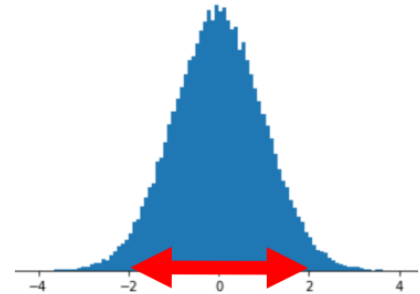


15.4.2. Normalverteilte Zufall

Die folgende Abbildung zeigt ein Histogramm. Die Daten wurde folgendermassen erzeugt: `rng.normal(loc=0, scale=1, size=10000)`

- Ungefähr wie viele Samples fallen in den rot markierten Bereich $[-2, +2]$

Die gegebene Normalverteilung hat den Mittelwert 0 und die Standardabweichung 1. Das Intervall $[-2, +2]$ ist somit um den Mittelwert zentriert, von -2 Standardabweichungen bis +2 Standardabweichungen. In diesem Intervall liegen rund 95% der Samples, also rund 9500



15.4.3. Wahrscheinlichkeiten, joint- Marginal-, Conditional- Probability

Suppose you receive Emails with text described in the following table. The table also shows if the emails are spam or not

No	Email Text	Spam
1	Hurry Sale Tomorrow	1
2	Rain tomorrow	0
3	Sale price tomorrow	1
4	tomorrow workshop rain	0

Calculate the following:

- The probability that any message in the above dataset is spam

$$\Pr(\text{spam}) = 0.5$$

2 von 4 Mails sind Spam $2/4=0.5$

- The probability that any message in the above dataset is not-spam

$$\Pr(\text{not-spam}) = 1 - 0.5 = 0.5$$

Ebenfalls 2/4 Man kann aber auch die "Gegenwahrscheinlichkeit" berechnen: $\Pr(\text{not-spam}) = 1 - \Pr(\text{spam}) = 1 - 0.5$

- Calculate the conditional Probabilities in the probability table below

Meaning of notation:

$\Pr(\text{Hurry} \mid \text{spam})$ = Given that the email is spam, what is the probability that the email contains "Hurry"?

$\Pr(\text{Hurry} \mid \text{not-spam})$ = Given that the email is not-spam, what is the probability that the email contains "Hurry"?

Beispiel: $\Pr(\text{Hurry} \mid \text{spam})$:

Es gibt zwei Spam Mails. Hurry kommt in einem dieser Spam Mails vor, deshalb: $\Pr(\text{Hurry} \mid \text{spam}) = 1/2 = 0.5$

Word	$\Pr(\text{word} \mid \text{spam})$	$\Pr(\text{word} \mid \text{not-spam})$
Hurry	$1/2 = 0.5$	$0/2 = 0$
Sale	$2/2 = 1$	$0/2 = 0$
Tomorrow	$2/2 = 1$	$2/2 = 1$
Rain	$0/2 = 0$	$2/2 = 1$
Workshop	$0/2 = 0$	$1/2 = 0.5$
Price	$1/2 = 0.5$	$0/2 = 0$

Hint: How often does the word occur in spam or not-spam emails?

d) Calculate the following probabilities:

$\Pr(\text{"Hurry"})$: what is the probability that an email contains "Hurry"?

Word	$\Pr(\text{word})$
Hurry	$1/4 = 0.25$
Sale	$2/4 = 0.5$
Tomorrow	$4/4 = 1$
Rain	$2/4 = 0.5$
Workshop	$1/4 = 0.25$
Price	$1/4 = 0.25$

e) Calculate the conditional probabilities in the following tables

$\Pr(\text{spam} \mid \text{"Hurry"})$: Probability that an email containing "Hurry" is a spam

Word	$P(\text{spam} \mid \text{word})$	$P(\text{not-spam} \mid \text{word})$
Hurry	$\frac{\Pr(\text{"hurry"} \mid \text{spam}) \Pr(\text{spam})}{\Pr(\text{"hurry"})} = \frac{0.5 \cdot 0.5}{0.25} = 1$	0
Sale	$\frac{1 \cdot 0.5}{0.5} = 1$	0
Tomorrow	$\frac{1 \cdot 0.5}{1} = 0.5$	0.5
Rain	$\frac{0 \cdot 0.5}{0.5} = 0$	1
Workshop	$\frac{0 \cdot 0.5}{0.25} = 0$	1
Price	$\frac{0.5 \cdot 0.5}{0.25} = 1$	0

Hint: Bayes Theorem

$$\Pr(\text{spam} \mid \text{word}) = \frac{\Pr(\text{word} \mid \text{spam}) \Pr(\text{spam})}{\Pr(\text{word})}$$

15.4.4. Lineare und Polynomiale Regression. Bias-variance trade-off

Sie bekommen folgende Daten und fitten ein kubisches

Modell an diese Daten:

- a) Formulieren Sie das Modell als mathematische

Gleichung $\hat{y} =$

$$\hat{y}_i = w_3 x_i^3 + w_2 x_i^2 + w_1 x_i + w_0$$

$$e_i = y_i - \hat{y}_i$$

$$\text{MSE} = (1 / 2N) \sum e_i^2$$

$$\text{MSE} = (1 / 2N) \sum (y_i - (w_3 x_i^3 + w_2 x_i^2 + w_1 x_i + w_0))^2$$

- b) Wie viele freie Parameter hat das Modell?

Das Modell hat 4 Parameter

- c) Formulieren Sie den MSE als mathematische

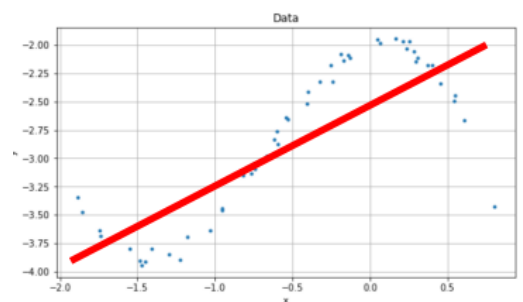
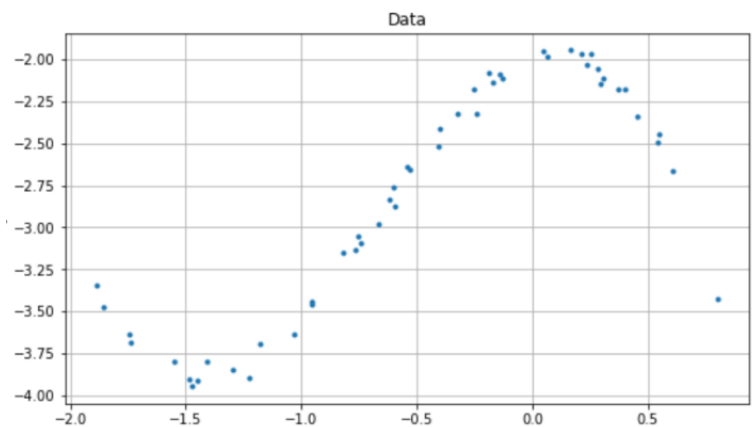
Gleichung. $\text{MSE} = \dots$ (siehe oben bei a))

-Anstelle von w kann natürlich auch β , θ , a , b , c , d , etc. verwendet werden.

-streng genommen ist der MSE nicht durch 2 geteilt. Es wird praktisch aber so gemacht, da sich beim Ableiten (für Optimierung mit SGD) die 1/2 mit der 2 aus $(\dots)^2$ kürzt

-Das Modell kann mit oder ohne den Subskript i geschrieben werden. Beides OK. Beim MSE braucht es aber einen Index (z.B. i) und die Summe muss von 1 bis N gehen.

-Die Parameter können auch anders nummeriert werden, gemischte Indices wie $w_1 x^3$ sind aber eher verwirrend



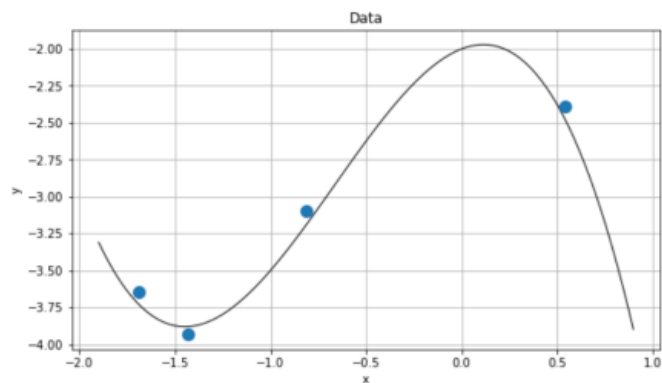
Nehmen Sie an, jemand würde ein einfaches lineares Modell (eine Gerade) an die oben gegebenen Daten fitten

- d) Skizzieren Sie das Ergebnis (die Gerade) direkt in die oben gegebene Figur.
e) Hat der Generalisierungsfehler des linearen Modells einen grossen oder kleinen Bias?
Das ist ein Beispiel für einen hohen/grossen Bias. (und tiefe Variance, underfitting).

- f) Nehmen Sie an, sie würden untenstehende vier Datenpunkte erhalten.

Ist es eine gute Idee, ein Polynom 3. Ordnung zu fitten?

Nein. Das kubische Modell hat vier Parameter. Wenn man mit nur vier Datenpunkten vier freie Parameter bestimmen will, findet man ein Polynom, welches exakt durch die Datenpunkte geht (der Training-Error wird 0, Overfitting). Da aber die Daten mutmasslich (in der Praxis immer) fehlerbehaftet sind (z.B. auch nur kleine Messfehler), muss man erwarten, dass ein «exakt auf die Fehler» gefittetes Modell schlecht generalisiert



15.4.5. SGD

Ein Modell mit zwei Parametern (a und b) wird auf Daten gefittet. Das Modell wird mit SGD optimiert.

Nach der Iteration t befindet sich der Optimizer an der Stelle (a=0.6, b=0.4).

An dieser Stelle werden die folgenden partiellen Ableitungen des Losses 'E' ausgewertet:

$$\partial E / \partial a = +0.20$$

$$\partial E / \partial b = -0.30$$

Nehmen Sie $\alpha = 0.1$ an.

- a) Berechnen Sie den Parametervektor zum Zeitpunkt t+1.

$$[a, b]_{t+1} = [a, b]_t - \alpha \cdot [\partial E / \partial a, \partial E / \partial b]$$

$$(0.6, 0.4) - 0.1 \cdot (0.2, -0.3) = (0.58, 0.43)$$

15.4.6. Logistic Regression

Suppose the task is to predict the admission decision of an applicant given his secondary school score (x1) and BS grades (x2).

Given some data and using logistic regression, we train a model that has the following parameters

$$\Pr(\text{accept} \mid x; w) = 1 / (1 + e^{-(w_1 \cdot x_1 + w_2 \cdot x_2 + w_3)})$$

$$w_1 = 1$$

$$w_2 = -0.7$$

$$w_3 = -0.7$$

- a) Calculate the probability that an applicant get i) rejected and ii) accepted if Secondary school score = 4.5 and BS grad = 5.5

```
1 linear_score = lambda x1, x2: x1-0.7*x2-0.7
```

```
1 linear_score(4.5,5.5)
```

```
-0.049999999999999996
```

```
1 np.exp(-linear_score(4.5,5.5))
```

```
1.0512710963760237
```

```
1 prob_accept = 1/(1+np.exp(-linear_score(4.5,5.5)))
```

```
1 prob_reject = 1-prob_accept
```

```
1 print(prob_accept)
```

```
2 print(prob_reject)
```

```
0.4875026035157898
```

```
0.5124973964842102
```

- b) Write down the formula for the Loss function used in logistic regression. Assume N data points

$$-\frac{1}{N} \sum_{i=1}^N y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

- c) Write the Loss (the formula, without calculating a value) for the single data point given above. You can plug-in the probability you found in a). Assume y=0 for rejected and y=1 for accepted (see below)

d) Calculate the Loss for the two possibilities: i) if the applicant was rejected ($y=0$) and ii) if the applicant was accepted ($y=1$)

$$\frac{-1}{N} \sum_{i=1}^N y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

For $N=1$, and plugging in the probabilities found above, we have:

$$-1 \cdot (y_i \cdot \log(0.4875) + (1 - y_i) \log(0.5125))$$

If the participant was rejected:

That means: what is the contribution of the data point ($x_i = (4.5, 5.5)$, $y_i = 0$) to the total loss?

$$\begin{aligned} \text{loss} &= -1 \cdot (0 + (1 - 0) \log(0.5125)) \\ &= 0.6685 \end{aligned}$$

If the participant was accepted:

That means: what is the contribution of the data point ($x_i = (4.5, 5.5)$, $y_i = 1$) to the total loss?

$$\begin{aligned} \text{loss} &= -1 \cdot (1 \cdot \log(0.4875) + 0) \\ &= 0.7185 \end{aligned}$$

Note that the loss is composed of two complementary terms. For each data point, only one of the two is evaluated, because either y_i or $1 - y_i$ equals 0.

Note that the loss is larger if the participant was accepted. This reflects well the fact that according to the model the participant has a (slightly) higher probability of being rejected.

15.5. THEORY QUESTION 2021 PART 2

15.5.1. Confusion Matrix für binäre Klassifikation

Ein Classifier wird mit $N=100$ Datenpunkten trainiert. Gegeben sind TP, FN und FP. Berechnen Sie die gesuchten Kennzahlen:

TP = 40 FN = 10
FP = 20 **TN = 30**

- Anzahl Datenpunkte mit Label «positiv» = $40+10=50$
- Anzahl Datenpunkte mit Label «negativ» = $20+30=50$
- Mean Accuracy = $(TP+TN)/N = (40+30)/100 = 0.7$
- Mean Error = $(FP+FN)/N = (20+10)/100 = 0.3$
- Precision = $TP/(TP+FP) = 40/(40+20) = 4/6 = 2/3 \approx 0.66$
- Sensitivity (=TPR) = $TP/(TP+FN) = 40/(40+10) = 4/5 = 0.8$
- Miss Rate (=FNR) = $1 - \text{TPR} = 0.2$
- False Positive Rate = $FP/(FP+TN) = 20/(20+30) = 2/5 = 0.4$
- Lässt sich die Qualität dieses Classifier in einer ROC-Kurve darstellen?

Man kann ($\text{FUR}=0.4$, $\text{TPR}=0.8$) im ROC-Raum darstellen. Das gibt dann **einen einzigen Punkt**, keine Kurve. Für eine Kurve müsste man den Classifier mit allen (vielen) Schwellenwerten (Threshold) im Intervall $[0, 1]$ auswerten

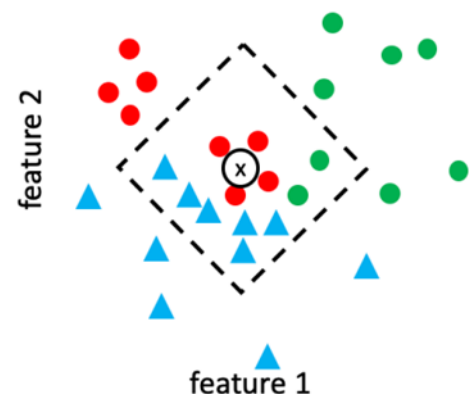
10) KNN

Betrachten Sie nachfolgende Daten. Sie werden mit KNN ($N=12$, Manhattan-Distanz) klassifiziert

- Klassifizieren Sie den neuen Datenpunkt x
[] rot [] grün **[x] blau** (6 neighbours)

- Ergänzen Sie (ungefähr) die Abbildung so, dass klar wird, wie Sie zu ihrem Resultat kommen.

- Das stehende Quadrat zeigt Punkte, welche dieselbe Manhattan-Distanz vom Punkt x haben
- Das Quadrat muss so gross sein, bis es genau $N=12$ Neighbours einschliesst
- Es wäre falsch einen Kreis zu skizzieren



Cross Validation und Regularisierung

Ein Machine-Learning Problem mit zwei unterschiedlichen Werten für den Regularisierungsparameter optimiert und mit Cross-Validation evaluiert. Die Ergebnisse stehen in folgender Tabelle. Analysieren Sie die Werte und beantworten Sie die Fragen

$\lambda=0.005$	Fold	performance score (higher = better)		
	1	95		
	2	30		
	3	70	sum	195
			avg	65
$\lambda=0.01$	Fold	performance score (higher = better)		
	1	80		
	2	75		
	3	65	sum	220
			avg	73.3

Welche der Antworten sind richtig (mehrere Antworten möglich):

- ☐ Man sollte das Modell mit dem Performance Score 95 verwenden
- ☒ Man sollte ein neues Modell trainieren
- ☒ Der Regularisierungsparameter 0.01 liefert das bessere Ergebnis
- ☒ Man sollte ein neues Modell auf allen Trainingsdaten mit $\lambda=0.01$ trainieren

Begründung:

Mit CV kann man die Generalisierungseigenschaften eines Modells abschätzen. Der durchschnittliche score ist im zweiten Fall besser $220/3 > 195/3$. Das bedeutet, dass eine stärkere Regularisierung ($\lambda=0.01$) im Durchschnitt bessere Ergebnisse liefert. Man würde nun ein neues Modell auf allen Trainingsdaten, mit $\lambda=0.01$ trainieren. Man kann noch ergänzen, dass $\lambda=0.01$ zu einem stärkeren Bias führt, dass aber die Varianz tiefer ist.

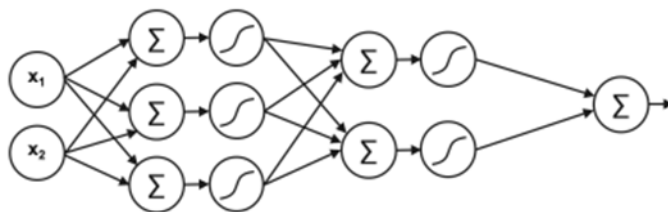
ANN

a) Skizzieren Sie ein ANN gemäß folgenden Angaben. Verwenden Sie die Darstellung aus der Vorlesung

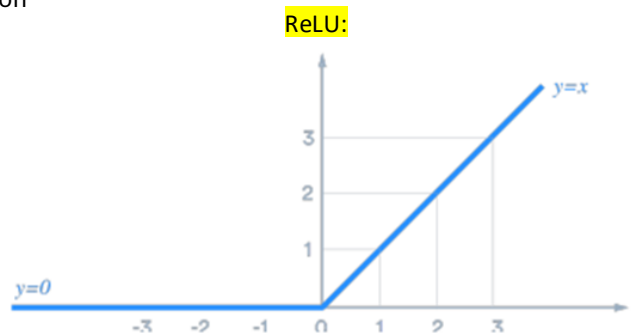
- zweidimensionales Input
- erster hidden Layer mit drei Neuronen
- zweiter hidden Layer mit einem Neuron und ohne Aktivierungsfunktion

b) Wieviele «trainable params» hat dieses Netz?

c) Skizzieren Sie den Funktionsgraphen der ReLU – Aktivierungsfunktion



erster hidden Layer: $3 \cdot (2 \text{ weights} + 1 \text{ bias}) = 9 \text{ parameter}$
 zweiter hidden Layer: $2 \cdot (3 \text{ weights} + 1 \text{ bias}) = 8 \text{ parameter}$
 output Layer: $1 \cdot (2 \text{ weights} + 1 \text{ bias}) = 3 \text{ parameter}$
 total: 20 Parameter



16. LÖSUNGEN ÜBUNGEN

16.1. ÜBUNG 2:

Assume an algorithm (for example word2vec) has found the following representations for the words "apple", "orange" and "bike":

apple	orange	bike
$\begin{bmatrix} 3 \\ 6 \\ 2 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 2 \\ 7 \\ 2 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 0 \\ 8 \\ 7 \end{bmatrix}$

Before calculating a value, guess the cosine similarity $SC(\text{"apple"}, \text{"orange"})$ and $SC(\text{"apple"}, \text{"bike"})$. apple and orange are similar -> SC close to 1. apple and bike are different -> SC is smaller but not 0

Calculate the cosine similarity $S_C(\text{"apple"}, \text{"orange"})$ and $S_C(\text{"apple"}, \text{"bike"})$.

```
# Define vectors
apple = np.array([3, 6, 2, 1])
orange = np.array([2, 7, 2, 0])
bike = np.array([1, 0, 8, 7])
new_word = np.array([0, 0.9, 12, 7])

# Calculate the length of the vector
length_apple = np.linalg.norm(apple)
print("length_apple={}".format(length_apple))

length_orange = np.linalg.norm(orange)
print("length_orange={}".format(length_orange))

dot_product = np.dot(apple, orange)
print("dot_product={}".format(dot_product))
cosine_similarity = dot_product / (length_apple * length_orange)
print("cosine_similarity={}".format(cosine_similarity))

length_apple=7.0710678118654755
length_orange=7.54983443527075
dot_product=52
cosine_similarity=0.9740492440449616
```

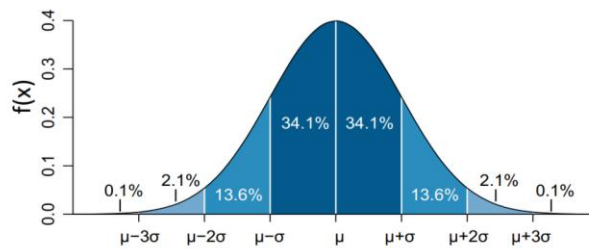
Why does it make sense to normalize the vector representations? How would the calculation simplify (become more efficient) if vectors are normalized, the denominator becomes $1 \times 1 = 1$. Calculating the cosine similarity reduces to the dot product. You normalize a vector once, in a preprocessing-step, later, you save computations each time you use that normalized vector in a cosine similarity calculation.

Let's assume that the representation was developed to classify words into fruits and

vehicles. Do you think the following vector represents a fruit or a vehicle? Calculate the cosine similarities. If the new word is similar to apple (and orange), it's likely a fruit. If it's more similar to bike, the new word will likely be classified a vehicle. From the calculations below, we see that the new vector is closer to the bike-vector than to the apple vector, and therefore probably represents a vehicle.

16.2. ÜBUNG 3

The following graph shows the PDF of a gaussian distribution with mean μ and standard-deviation σ . Theoretically, a gaussian random-variable has a range from $-\infty$ to $+\infty$, but the probability to take a value far from the mean goes quickly to very small values (practically 0).



https://commons.wikimedia.org/wiki/File:Standard_deviation_diagram_micro.svg

For continuous random variables, we cannot read the probability of events directly from the probability density function; the values on the y-axis are **not** probabilities! Instead, the "area under the curve" (= the integral) corresponds to the likelihood of an event to fall into an interval $[a, b]$:

$$Pr(a < X < b) = \int_a^b f(x)dx$$

- a) A random variable X follows a gaussian distribution. What is the probability for X to take a value in the interval $[\mu-2\sigma, \mu+2\sigma]$

Formally, we are interested in the area under the curve in the given interval:

$$Pr(\mu - 2\sigma < X < \mu + 2\sigma) = \int_{\mu-2\sigma}^{\mu+2\sigma} f(x)dx$$

We can simply approximate that area using the values in the figure above:

$$13.6 + 34.1 + 34.1 + 13.6 \approx 95.4\%$$

- b) A Gaussian random variable Y has mean $\mu=45$ and $\sigma=10$.

- i) What is the probability that Y takes a value < 45

By symmetry, this is 50%

- ii) What is the probability that Y takes a value between 35 and 65 ?

You should note that this interval can be expressed in terms of mean μ and std-dev σ :

$$Pr(\mu - 1\sigma < Y < \mu + 2\sigma) \approx 34.1 + 34.1 + 13.6 \approx 81.8\%$$

- iii) What is the probability that Y takes the (exact) value 45?

$$\int_{x=45}^{x=45} f(y)dy = 0$$

The area under the "curve" is 0!

x_1	x_2	y
1	2	2
2	3	3
3	4	5

16.3. ÜBUNG 8: REGULARIZATION

Consider the linear regression model $y = w_1 * x_1 + w_2 * x_2$, where w_2 and w_1 are the weights (coefficients), x_1 and x_2 are the features, and y is the target variable. Let's say we have the following dataset: And we have initial weights $w_1 = 0.5$ and $w_2 = 0.5$.

16.3.1. MSE

Calculate the Mean Squared Error (MSE) loss. The formula for MSE is:

$$MSE = 1/N * \sum (y_i - (w_1 * x_{1i} + w_2 * x_{2i}))^2$$

Where y_i is the actual output, $(w_1 * x_{1i} + w_2 * x_{2i})$ is the predicted output ($y^{\wedge}$), and N is the number of data points.

First, calculate the predicted output for each data point:

$$\hat{y}_1 = 0.5 * 1 + 0.5 * 2 = 1.5$$

$$\hat{y}_2 = 0.5 * 2 + 0.5 * 3 = 2.5$$

$$\hat{y}_3 = 0.5 * 3 + 0.5 * 4 = 3.5$$

Then, calculate the MSE loss:

$$loss = 1/3 * ((2 - 1.5)^2 + (3 - 2.5)^2 + (5 - 3.5)^2)$$

$$loss = 1/3 * ((0.5)^2 + (0.5)^2 + (1.5)^2)$$

$$loss = 1/3 * (0.25 + 0.25 + 2.25)$$

$$loss = 1/3 * 2.75$$

$$loss = 0.9167$$

16.3.2. L1 (Lasso)

The L1 regularization adds the absolute value of the magnitude of the coefficients as penalty term to the loss function. Where λ is the regularization parameter. Let's say $\lambda = 0.1$, compute the L1 regularization for the given weights.

$$L1 = loss + 0.1 * (|0.5| + |0.5|)$$

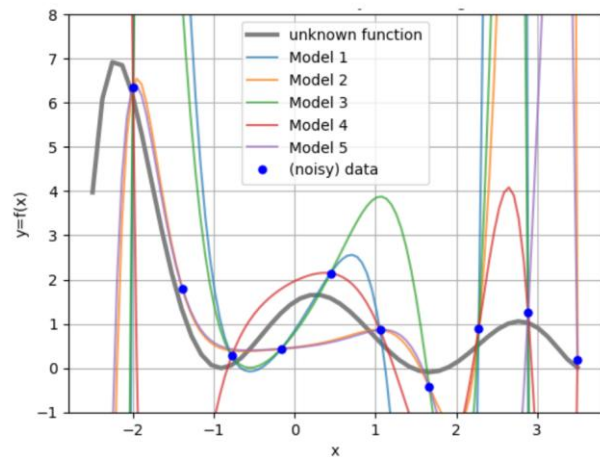
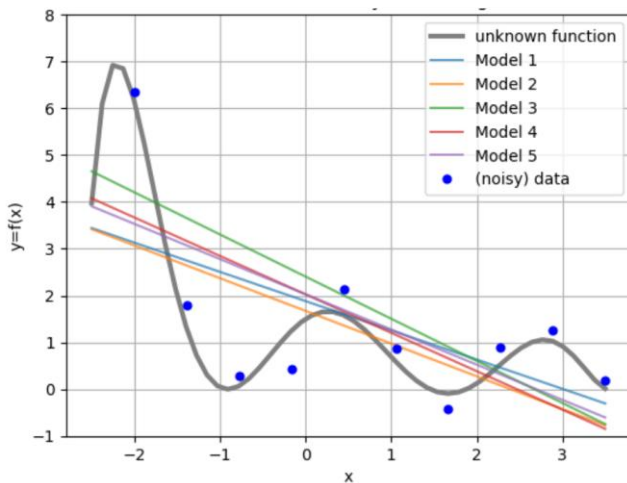
$$L1 = loss + 0.1 * (1.0)$$

16.3.3. L2 (Ridge)

The L2 regularization adds the square of the magnitude of the coefficients as penalty term to the loss function. Again, let's say $\lambda = 0.1$, compute the L2 regularization for the given weights. $L2 = loss + 0.1 * ((0.5)^2 + (0.5)^2)$

16.3.4. Polynomial Features

You are given a training dataset of 10 samples drawn from an unknown underlying function. The samples are shown in blue. Your task is to randomly select 7 data points as training data and fit a linear regression model to them. You repeat this 5 times, randomly choosing the training data. Sketch plausible plots showing the model output illustrating how the model fit the training data.



Now, instead of linear regression, use PolynomialFeatures with degree $M = 10$ to fit models of higher complexity to the same training dataset. Repeat the procedure 5 times: randomly select 7 data points as training data, fit the model, and obtain the resulting functions. Sketch plausible plots showing these fitted models.

Explain the effect of model complexity on bias and variance trade-off in the above scenario. We see that simple linear regression model has high bias and low variance. As the model complexity increases, the model has low bias and very high variance.

16.4. ÜBUNG 9

16.4.1. Standardization

Consider a 1-dimensional dataset with 4 points: $X = [1, 1, 5, 5]$. Compute the standardized values using the following two schemes:

a.) Min-Max scaling to the range $[a, b]$ where a and b forms the desired scaling range with $a=0, b=1$.

b.) Standardization to variance 1 (z-score scaling/standardization)

$$x' = \frac{x - X_{\min}}{X_{\max} - X_{\min}}$$

$$x_{\text{scaled}} = x' * (b - a) + a$$

$$\begin{aligned} X_{\min} &= 1 \\ X_{\max} &= 5 \end{aligned}$$

$$x'_1 = x'_2 = \frac{1 - 1}{5 - 1} = 0$$

$$x'_3 = x'_4 = \frac{5 - 1}{5 - 1} = 1$$

$$\mathbf{X}_{\text{scaled}} = [0, 0, 1, 1]$$

$$z = \frac{(x - \mu)}{\sigma}$$

$$\begin{aligned} \text{where } \mu &= \text{mean}(X) = \frac{1}{n} \sum_{i=1}^n x_i \\ \text{Standard deviation } \sigma &= \sqrt{\frac{1}{n} \sum (x_i - \mu)^2} \end{aligned}$$

$$n = 4$$

$$\mu = \frac{1 + 1 + 5 + 5}{4} = 3$$

$$\sigma = \sqrt{\frac{1}{4} ((1 - 3)^2 + (1 - 3)^2 + (5 - 3)^2 + (5 - 3)^2)} = \sqrt{\frac{1}{4} (4 + 4 + 4 + 4)} = 2$$

$$x'_1 = x'_2 = \frac{1 - 3}{2} = -1$$

$$x'_3 = x'_4 = \frac{5 - 3}{2} = 1$$

$$\mathbf{Z} = [-1, -1, 1, 1]$$

The regression model is: $\hat{y} = w_1 * x + w_0$

We evaluate the models using the $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

You are given the following dataset

x	y
1	2
2	4
3	6
4	7

Use the following fold split:

Fold 1: (1,2), (2,4)

Fold 2: (3,6), (4,7)

16.4.2. Cross-Validation

We will perform 2-fold cross-validation for a simple linear regression model.

• Model 1: Trained on Fold 1 and validated on Fold 2 • Model 2: Trained on Fold 2 and validated on Fold 1. Draw the datapoints and sketch the resulting fitted linear regression models (lines). You can then read the value for $\hat{y}_i$ from the plot. Calculate the MSE for both models separately. Calculate the 2-fold cross validation error.

$$\text{CV-MSE} = \frac{\text{MSE}_1 + \text{MSE}_2}{2} = \frac{0.5 + 2.5}{2} = 1.5$$

$$\text{MSE}_1 = \frac{1}{2} ((6 - 6)^2 + (7 - 8)^2) = \frac{1}{2} (0 + 1) = 0.5 \quad \text{MSE}_2 = \frac{1}{2} ((2 - 4)^2 + (4 - 5)^2) = \frac{1}{2} (4 + 1) = 2.5$$

16.5. ÜBUNG 10

Suppose the task is to predict the admission decision of an applicant given his secondary school score (x_1) and BS grades (x_2). Given some data and using logistic regression, we train a model that has the following parameters

$$p(x) = \frac{1}{1 + e^{-(w_1 x_1 + w_2 x_2 + w_3)}}$$

1. Estimate the probability that an applicant gets accepted if Secondary school score = 4.5 and BS grade = 5.5

$$\begin{aligned} w_1 &= 1 \\ w_2 &= -0.7 \\ w_3 &= -0.7 \end{aligned}$$

2. What should be the BS grade of an applicant with secondary school score = 4.5 to have a 50 % chance of getting accepted?

$$z = (w_1 x_1 + w_2 x_2 + w_3)$$

$$z = 1 \cdot 4.5 + (-0.7) \cdot 5.5 + (-0.7) = -0.05$$

$$p(x) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{0.05}} = \frac{1}{1 + 1.05} = 0.488$$

$$p(x) = 0.5 = \frac{1}{1 + e^{-z}} \text{ substitute: } e^{-z} \rightarrow x$$

$$0.5 = \frac{1}{1 + x} \Rightarrow x = 1 = e^{-z}$$

$$e^{-z} = 1$$

$$\ln(e^{-z}) = \ln(1)$$

$$-z = 0 \Rightarrow (w_1 x_1 + w_2 x_2 + w_3) = 0$$

$$x_2 = \frac{0 - w_1 x_1 - w_3}{w_2}$$

$$x_2 = \frac{0 - 1 \cdot 4.5 - (-0.7)}{-0.7} = 5.43$$

16.6. ÜBUNG 11

16.6.1. Logistic Regression Decision Boundary

Given some data and using logistic regression, we train a model that has the following parameters

$$p(x) = \frac{1}{1 + e^{-(w_1 x_1 + w_2 x_2 + w_3)}}$$

$$\begin{aligned} w_1 &= 1 \\ w_2 &= 0 \\ w_3 &= 7 \end{aligned}$$

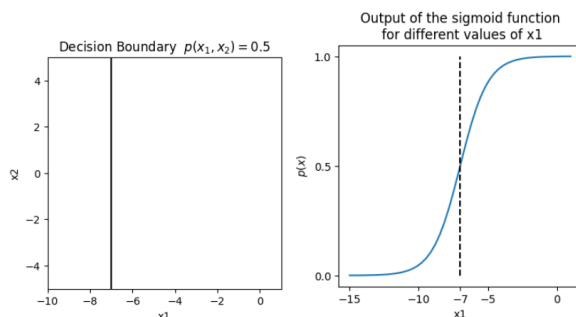
$$p(x) = 0.5 = \frac{1}{1 + e^{-z}}$$

$$0.5 = \frac{1}{1 + e^{-z}} \Rightarrow e^{-z} = 1 \Rightarrow z = 0$$

$$z = 0 \Rightarrow (w_1 x_1 + w_2 x_2 + w_3) = 0$$

$$(1 \cdot x_1 + 0 \cdot x_2 + 7) = 0 \Rightarrow x_1 = -7$$

Since $w_2 = 0$, the decision boundary is parallel to the x_2 axis at $x_1 = -7$



$$d_M(x_1, x_2) = \sum_{i=1}^p |x_{1,i} - x_{2,i}|$$

Example for $d_M(5,1)$

$$d_M(5,1) = (3 - 2) + (3 - 2) = 2$$

Manhattan distances	
$d_M(5,1)$	2
$d_M(5,2)$	2
$d_M(5,3)$	2
$d_M(5,4)$	2

Manhattan distances	
$d_M(1,2)$	4
$d_M(1,3)$	2
$d_M(1,4)$	2
$d_M(1,5)$	2

k = 3, metric = Manhattan			
Train points	Val point	true class	predicted class
1, 2, 3, 4	5	-	+
1, 2, 3, 5	4	+	+
1, 2, 4, 5	3	+	+
1, 3, 4, 5	2	+	+
2, 3, 4, 5	1	+	+

Accuracy = $(1/5) * (0 + 1 + 1 + 1 + 1) = 0.8$ i.e. 80%

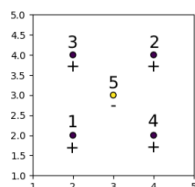
In this case, the choice of distance metric does not affect the classification with $k = 3$.

16.6.2. K-NN Classifier

We have a small dataset with 2 features. The points are (2,2), (4,4), (2,4), (4,2), and (3,3). The points (2,2), (4,4), (2,4), (4,2) belong to +ve class. The point (3,3) belongs to -ve class.

1. We now build a K-NN classifier using Euclidean distance as the distance metric. We want to evaluate the classifier with LOOCV (Leave One Out Cross Validation). Calculate the accuracy for LOOCV for $k=1$, and $k=3$.

2. Suppose we change the distance metric to Manhattan distance. Do you see any effect on classification with $k=3$?



Euclidean distances	
$d_E(5,1)$	$\sqrt{2}$
$d_E(5,2)$	$\sqrt{2}$
$d_E(5,3)$	$\sqrt{2}$
$d_E(5,4)$	$\sqrt{2}$

Euclidean distances	
$d_E(1,2)$	$\sqrt{8}$
$d_E(1,3)$	2
$d_E(1,4)$	2
$d_E(1,5)$	$\sqrt{2}$

k = 1, metric = Euclidean			
Train points	Val point	true class	predicted class
1, 2, 3, 4	5	-	+
1, 2, 3, 5	4	+	-
1, 2, 4, 5	3	+	-
1, 3, 4, 5	2	+	-
2, 3, 4, 5	1	+	-

Accuracy = $0 / 5 = 0.0$

k = 3, metric = Euclidean			
Train points	Val point	true class	predicted class
1, 2, 3, 4	5	-	+
1, 2, 3, 5	4	+	+
1, 2, 4, 5	3	+	+
1, 3, 4, 5	2	+	+
2, 3, 4, 5	1	+	+

Accuracy = $4 / 5 = 0.8$ i.e., 80%

$$d_E(x_1, x_2) = \sqrt{\sum_{i=1, p} (x_{1,i} - x_{2,i})^2}$$

Example for $d_E(5,1)$

$$d_E(5,1) = \sqrt{(3 - 2)^2 + (3 - 2)^2} = \sqrt{2}$$

16.7. ÜBUNG 13: ENSEMBLE

16.7.1. Hard voting with weights

An AI-assisted Email classification system uses Ensemble of three classifiers: 1. Logistic regression `log_clf`, 2. 3-Nearest neighbour `knn_clf3` and 3. 10-Nearest neighbour to predict class spam (class =1), ham (class =0). The Ensemble uses hard voting with weights. The weights are defined as [0.1, 0.3, 0.6] respectively. For a particular email, the predictions of the three classifiers are [ham, ham, spam], respectively. What is the final class predicted by the Ensemble?

Hard voting is a method of combining the predictions of multiple classifiers into a single prediction. In hard voting, the predictions of each classifier are summed for each class label, and the class label with the most votes is chosen as the final prediction. When using weights in hard voting, each vote is weighted according to the specified weight. For this example, we have: *The weighted score for ham = 0.1 + 0.3* *The weighted score for spam = 0.6* So we get: *ham: 0.4 , spam: 0.6* Since the weight of the third classifier, which is predicting spam, is greater than the combined weights of the other two classifiers, the final prediction of the Ensemble would be spam.

16.7.2. Ensemble Soft Voting

For spam-ham classification problem, we have three classifiers (`clf1`, `clf2`, `clf3`) and the ensemble uses soft voting. For an Email, the classifiers return the following probabilities (`predict_proba`): $Pr_clf1(Spam) = 0.6$, $Pr_clf2(Spam) = 0.8$, $Pr_clf3(Spam) = 0.2$ Will the Ensemble classify the Email as Spam or Ham?

In this case, the three classifiers have predicted the following probabilities for the spam class: [0.6, 0.8, 0.2]. When these probabilities are averaged, the resulting average for spam is $(0.6 + 0.8 + 0.2) / 3 = 0.53$. Since this average is greater than 0.5, the Ensemble would classify the email as spam.

16.7.3. Ensemble Soft Voting with weights

For spam-ham classification problem, we have two classifiers (`clf1`, `clf2`) and we use an ensemble method soft voting with weighted averages. The weights for the classifiers are $w_clf1 = 1.2$, $w_clf2 = 0.8$ For an Email, $Pr_clf1(Spam) = 0.8$, $Pr_clf2(Spam) = 0.4$ Will the ensemble classify the Email as Spam?

In this case, the two classifiers have predicted the following probabilities for the spam class: [0.8, 0.4], and the corresponding weights are [1.2, 0.8]. When these probabilities are multiplied by their corresponding weights and then averaged, the resulting weighted average is $(0.8 * 1.2 + 0.4 * 0.8) / 2 = 0.64$. Since this weighted average is greater than 0.5, the Ensemble would classify the email as spam.

16.7.4. Ensemble Soft Voting with weights

We are training AdaBoost on a dataset of size 100. A weak learner that we have trained classifies 68 out of 100 samples correctly. What is the predictor weight that AdaBoost will assign to this predictor?

First, we assign a sample weight to each sample. *Sample weight = 1 / # of samples = 0.01* In this case, the predictor has classified 68 out of 100 samples correctly. We now need the accuracy of this classifier. *accuracy = 68 * 0.01 = 0.68* The predictor weight is then calculated by the following formula. *predictor weight = $\ln(1 - accuracy)$* So, for this instance we get *predictor weight = $\ln(1 - 0.68)$, predictor weight = 0.754*

16.8. ÜBUNG 14

16.8.1. Softmax

A deep neural network classifies images of animals as one of these four classes: cat, dog, bird, or fish.

For a given image, the network outputs the following scores-vector $\vec{s}_k = [1, 6, 2, 3]$.

We can apply the softmax function to this vector to transform these scores into probabilities.

Apply the softmax function given by this formula and calculate the probabilities.

- $p_k = \frac{\exp(s_k(x))}{\sum_{j=1}^M \exp(s_j(x))}$
- M is the total number of classes
- $s_k(x)$ is the score for each class k

$$\begin{aligned}\exp(s_1(x)) &= e^1 = 2.718 \\ \exp(s_2(x)) &= e^6 = 403.429 \\ \exp(s_3(x)) &= e^2 = 7.389 \\ \exp(s_4(x)) &= e^3 = 20.086 \\ p_1 &= 2.718/433.622 = 0.006 \\ p_2 &= 403.429/433.622 = 0.93 \\ p_3 &= 7.389/433.622 = 0.017 \\ p_4 &= 20.086/433.622 = 0.046\end{aligned}$$

Calculate the exponential of each score:

$$[0.006, 0.93, 0.017, 0.046]$$

Calculate the sum of all the exponentials: $\sum_{j=1}^{M=4} \exp(s_j(x)) = 2.718 + 403.429 + 7.389 + 20.086 = 433.622$

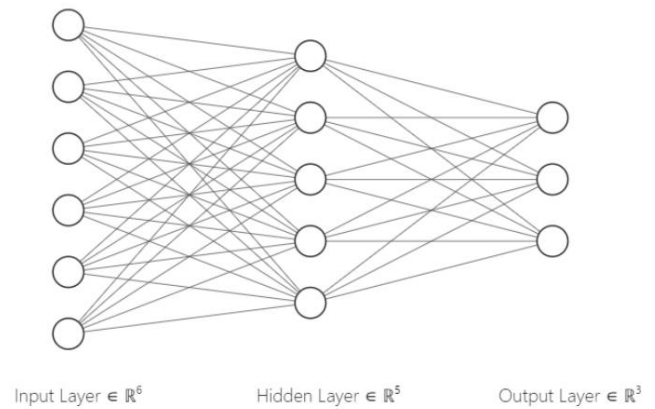
Divide each exponential by the sum of all exponentials to get the final output:

The output of the softmax function can be interpreted as a probability distribution over the set of scores that were input to the function. The values in the output array sum to 1, and each value represents the probability that the image belongs to that class.

16.8.2. Number of parameters

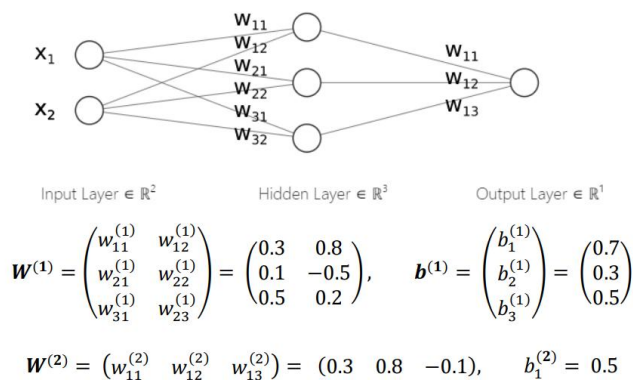
You train a neural network with an input layer with 6 units, a hidden layer with 5 units and an output layer with 3 units. Calculate how many trainable parameters this neural network has.

The input layer just represents the input vector of length 6 and has no trainable parameters. The hidden layer has for each unit a weight for every unit in the input layer plus a bias parameter. $number_of_parameters_hidden = (6 * 5) + 5 = 35$ The output layer has for each unit a weight for every unit in the hidden layer plus a bias parameter $number_of_parameters_output = (5 * 3) + 3 = 18$ So the total number of trainable parameters is: $number_of_parameters_total = 35 + 18 = 53$



16.8.3. Forward Pass

Calculate the output of this given neural network, when inputting $\mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$



$$ReLU(x) = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise} \end{cases}$$

Forward pass is given by
 pre-activation hidden layer: $\mathbf{z}^{(1)} = \mathbf{W}^{(1)} * \mathbf{x} + \mathbf{b}^{(1)}$
 activation hidden layer: $\mathbf{h}^{(1)} = ReLu(\mathbf{z}^{(1)})$
 pre-activation output layer: $\mathbf{z}^{(2)} = \mathbf{W}^{(2)} * \mathbf{h}^{(1)} + \mathbf{b}^{(2)}$
 model output: $\hat{\mathbf{y}} = ReLu(\mathbf{z}^{(2)})$

SOLUTION:

$$\mathbf{z}^{(1)} = \begin{pmatrix} 0.3 & 0.8 \\ 0.1 & -0.5 \\ 0.5 & 0.2 \end{pmatrix} * \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0.7 \\ 0.3 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 2.6 \\ -0.6 \\ 1.4 \end{pmatrix}$$

$$\mathbf{h}^{(1)} = ReLu\left(\begin{pmatrix} 2.6 \\ -0.6 \\ 1.4 \end{pmatrix}\right) = \begin{pmatrix} 2.6 \\ 0 \\ 1.4 \end{pmatrix}$$

$$\mathbf{z}^{(2)} = (0.3 \quad 0.8 \quad -0.1) * \begin{pmatrix} 2.6 \\ 0 \\ 1.4 \end{pmatrix} + 0.5 = 1.14$$

$$\hat{\mathbf{y}} = ReLu(1.14) = 1.14$$

Instead of the matrix calculations, one can also solve the equations for each neuron one by one.

This is probably easier to understand but not computationally efficient.

Output of neuron 1 in the hidden layer 1 is

$$\begin{aligned} h_1 &= Relu(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}) \\ h_2 &= Relu(w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}) \\ h_3 &= Relu(w_{31}^{(1)} x_1 + w_{32}^{(1)} x_2 + b_3^{(1)}) \end{aligned}$$

Final Output

$$\hat{\mathbf{y}} = ReLu(w_{11}^{(2)} h_1 + w_{12}^{(2)} h_2 + w_{13}^{(2)} h_3 + b_1^{(2)}) = 1.14$$